

Assessment Task 2

FormBridge Restaurant Enquiry Workspace

36118 Applied Natural Language Processing

Student Name: Karthik Ramesh
Student ID: 26043455

Abstract

FormBridge turns changing restaurant conversations into booking briefs that staff can inspect, correct and share. It tracks five fields, preserves uncertainty and links values to supporting messages. The web application includes guest intake, voice transcription, guest-care suggestions, private review links and a team desk. Staff control approval; messaging and reservation connections are simulated.

On 20 inspected development dialogues from the Schema-Guided Dialogue dataset, rules, direct contextual extraction and FormBridge achieved 35/100, 89/100 and 97/100 canonical field matches. FormBridge completed 17/20 briefs and required approximately twice the direct method's mean replay time. Its separate regression run matched 100/100 fields on 20 inspected test dialogues. These results do not estimate unseen performance.

Applying validation to identical proposals reduced matches from 88/100 to 85/100 in a controlled comparison, exposing overly restrictive checks. Twenty constructed cases passed checks for values, uncertainty, clarification and approval, including required sources for 93/93 populated fields. Verification also includes 230 automated tests, production checks and an eight-task live walkthrough. No recruited-user study or restaurant productivity measurement was conducted.

Contents

Section	Page
1 Problem and objectives	3
2 Product and user journey	4
3 Architecture and state	5
4 NLP methods and choices	6
5 Data and preparation	7
6 Evaluation design	8
7 Results and resource use	9
8 Error analysis and development findings	10
9 Interaction design	11
10 Functional verification	12
11 Ethics and limitations	13
12 Project delivery and next steps	14
References	15
Appendices A to F	16–21

1 Problem and objectives

The restaurant enquiry problem

A guest asks for four people at seven, then changes the request to five people sometime after eight. The current guest count is clear; the exact time is not. Reading only the first message produces an outdated brief. Reading only the last one loses the restaurant and date. Choosing 8:30 pm would add a decision the guest has not made.

This is a dialogue state tracking problem: maintaining the current values and constraints across a changing conversation. Earlier research, including TRADE, predicts state values from utterances rather than relying entirely on a fixed list of possible values (Wu et al., 2019). The Schema-Guided Dialogue dataset supplies service descriptions, task-oriented conversations and annotated states for this problem (Rastogi et al., 2020).

FormBridge applies this task to small restaurant teams. Front-of-house staff need to understand an enquiry, ask for missing details and hand it to the next person without losing a revision or special request. Guests need a clear way to describe their plans and correct them. The project defines these needs through enquiry scenarios; it has not established market demand through restaurant interviews.

Objectives and research question

The principal objective was to build a usable web application around accurate, inspectable booking information. Four technical objectives guided implementation:

1. Maintain restaurant, city, date, time and guest count as messages change.
2. Explain extracted values through source quotations, preserve uncertainty and require staff review.
3. Compare a traditional rule baseline with contextual extraction under a documented evaluation procedure.
4. Carry an enquiry through intake, review and team handoff, with protected records and recoverable errors.

Research question: How does contextual extraction with evidence validation and state reconciliation compare with rules and direct extraction, and how can its output support staff review of restaurant enquiries?

Scope and completion criteria

The implemented product accepts English text and short voice messages, with a maximum of 30 transcript messages of 1,000 characters each. Optional guest-care details sit alongside the five booking fields. Staff can save cases locally or in a private team, assign work, return a review link and prepare a handoff.

A successful workflow produces a brief whose current values, unresolved questions and staff decisions are visible. Approval means that the brief has been reviewed. External email, WhatsApp and reservation actions run through mock connectors; no real booking, payment or availability guarantee is made. The product scope is broader than the five-field benchmark, so workflow verification is reported separately from extraction accuracy.

2 Product and user journey

FormBridge has two connected entry points. Staff start with an empty booking desk and enter their own transcript, import a saved enquiry or review a voice transcription. Restaurants can publish a branded guest page or embed its assistant on their website. The assistant uses approved restaurant information and gathers the request before the guest consents to share contact details with the team.

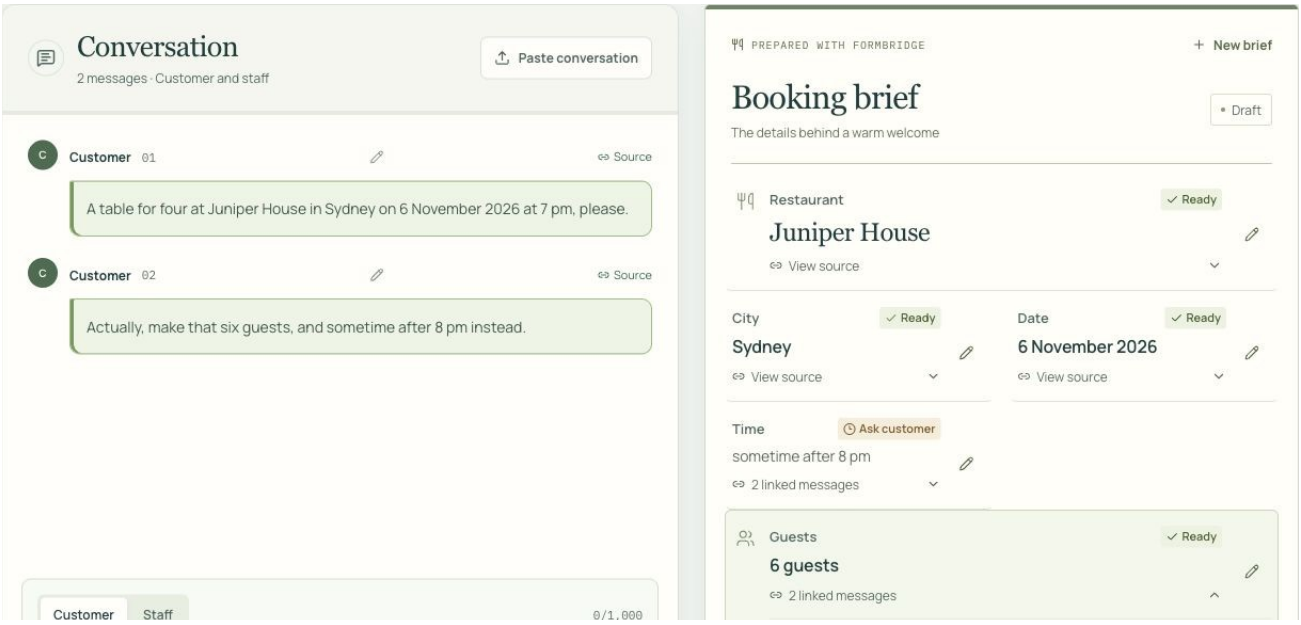


Figure 1. The live booking desk after a customer changes four guests to six and replaces 7 pm with “sometime after 8 pm”. The count is resolved; the exact time still needs an answer. Original product capture, 13 October 2026.

A complete enquiry

In Figure 1, the restaurant, city and date survive the correction. Staff can inspect the supporting messages and change history, then ask for an exact time. A proposed 8:30 pm becomes resolved only after the customer accepts it. Approval then applies to the reviewed brief.

The shared workflow adds contact details, accepted guest-care requests, an owner and a due time. A private review link lets the guest request a correction. That change becomes a customer message, clears approval and returns the enquiry to **Needs reply**. A simulated reservation connection demonstrates transfer and failure handling without reserving a table.

Work beyond the first reply

The desk supports comments, priority, ownership, returning-guest history and exact reminders. Staff explicitly link or merge guest records; the system does not assume that matching names identify the same person. A service planner organises saved enquiries with resolved dates. The desk assistant can draft replies, suggest a follow-up and prepare a handoff note, all subject to staff acceptance.

Device sign-in and private team invitations work in the deployed product. Email signup and password recovery can be explored in a separate, labelled simulation. Real email delivery remains disabled until a sender is configured and tested.

3 Architecture and state

The application uses Next.js 16.3.6, React 19.3.0 and TypeScript 5.9.3 on Vercel. OpenAI SDK 7.23.0 provides language and transcription requests; Zod 4.6.5 validates inputs and structured outputs. Supabase provides authentication, PostgreSQL storage and row-level security. A locked dependency file records the installed versions.

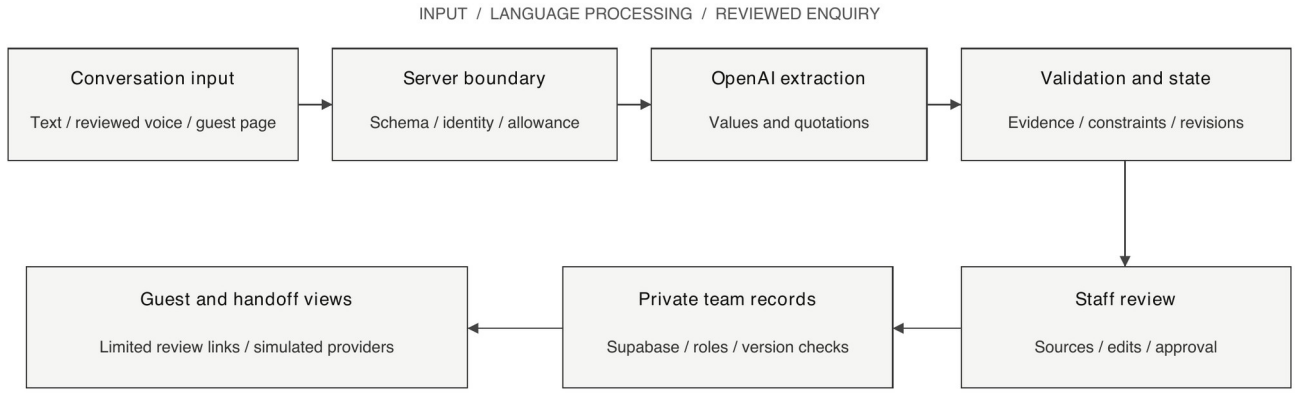


Figure 2. Application architecture. Model output is checked before it changes the brief. Shared workspace reads and changes require membership; the public booking desk works independently. External messaging and reservations use simulations.

State ownership

The model proposes a current value, status, constraint and evidence for each field. Application code decides what enters the brief. A field may be missing, resolved or awaiting clarification, and may originate from the model or a staff edit. Source IDs connect it to transcript messages. Revision history records changes; approval refers to one particular revision. A changed request therefore cannot retain an earlier approval silently.

The active draft uses browser session storage. The device library uses local storage and holds up to 50 cases. Shared cases are saved in Supabase with workspace membership checks. Guest preferences, comments and follow-up actions remain distinct from the five extracted booking fields. Reimported JSON and older model-derived drafts require renewed checking before approval.

Shared edits and request boundaries

Writes carry a saved version. A stale write returns a conflict rather than overwriting a teammate's changes. Where possible, the interface preserves unrelated edits and asks which version to keep when the same field changed. In-flight reply requests are tied to the selected case, preventing a late response from appearing in another enquiry.

Server routes validate body size and field schemas before provider calls. A shared database allowance reserves estimated usage, then settles known usage; uncertain failed requests retain their reservation. If the allowance service is unavailable in production, AI requests stop while manual review and saved work remain available. This is an application spending control, not an account-wide billing guarantee.

4 NLP methods and choices

Three approaches to booking state

Table 1. Approaches to booking-state extraction.

Approach	Method	Purpose
Rules	Regular expressions, number-word conversion and simple correction or acceptance patterns	An interpretable baseline without model calls
Direct extraction	A structured five-field prediction from each visible conversation prefix	Measures contextual extraction with minimal application reconciliation
FormBridge	Contextual candidates with quotations, deterministic validation and revision-aware reconciliation	Produces a state that staff can inspect and review

All approaches receive the same speaker-labelled prefixes. FormBridge reconstructs the request, then checks message IDs, literal quotations, value support, customer acceptance and recency. It checks dates, time constraints, guest counts and cancellation. Manual edits are preserved; later conflicts require review.

Structured output makes the response shape predictable; it does not make the content correct (OpenAI, n.d.-b). A phrase such as “after 7 pm” must remain a constraint, and a time's numeral must not become a guest count. Validation addresses these task-specific errors. Its pattern-based checks can still reject a valid paraphrase or misunderstand acceptance, so human inspection remains necessary.

Model and latency choices

The deployed configuration uses `gpt-6-luna` with one `gpt-6.1-sol` repair or transient-failure recovery. Both use low reasoning effort; SDK retries are disabled. Output limits are 2,400 tokens for extraction with evidence and 900 for direct extraction.

A ten-dialogue development trial with Luna matched 45/50 fields, with no failed dialogues and a mean replay time of 27.78 seconds. A later Sol-primary/Astra-repair trial on the same IDs matched 38/50, failed one dialogue and averaged 52.39 seconds over the nine completed dialogues. Prompts and validation also changed between trials, so this does not isolate model quality. It supports the practical choice of Luna for ordinary requests; Appendix E gives the trial records.

Sol is reserved for one further check or recovery when needed. Its benefit was not isolated in a controlled repair-model trial. The 12- and 14-second limits leave time for application work within the 30-second route limit. Hosted models avoid training and serving weights, at the cost of network dependence, external processing and model aliases that may change.

Language features around the brief

Voice input uses `gpt-transcribe`; staff review the transcript before adding it. Guest-care suggestions require customer quotations and individual staff acceptance. The desk assistant proposes notes or follow-ups without booking tables or overwriting fields. Restaurant questions retrieve matching categories of approved facts and acknowledge missing answers.

5 Data and preparation

Dataset and mapping

The evaluation uses Google's Schema-Guided Dialogue dataset (Google Research, n.d.). The exact corpus revision is specified in Appendix B. Conversations are retained only when they concern one restaurant service and contain a customer frame with the `ReserveRestaurant` intent. The following subset was identified before applying the maximum-prefix-length filter.

Table 2. Audited restaurant subset before prefix-length filtering.

Split	Restaurant service	Eligible dialogues	Reservation customer turns
Train	Restaurants_1	367	1,870
Development	Restaurants_2	73	385
Test	Restaurants_2	73	342

The training schema uses `city` and `party_size`; the development and test schemas use `location` and `number_of_seats`. The loader maps both to the application's city and guest-count fields. Restaurant name, date and time complete the five-field task. No model weights were trained on these data.

Preparing each example

Files are read in a stable order. The input ends at the final reservation customer turn and is restricted to at most 30 messages. USER and SYSTEM become customer and staff, with stable message IDs. The final annotated slot values supply acceptable answers for scoring. Each system processes every customer prefix in chronological order, so later messages are unavailable at earlier steps.

Gold labels, schema defaults and future turns are excluded from model input. Historical conversations have no reference date. Consequently, “tomorrow” is not interpreted using the date of the experiment. A yearless date can remain a recognised value while still requiring clarification before staff approval.

Other inputs and data limitations

Live inputs are user-provided conversations, approved restaurant facts and reviewed voice transcripts. Product screenshots and functional tests use fictional restaurants and guests. These are separate from SGD and are not collected restaurant traffic. Guest-care and workflow features are outside the dataset's five-field labels.

SGD provides useful task-oriented state labels, but does not fully label evidence quality, clarification wording, staff workload or operational readiness. Its public availability also means absence from a hosted model's pretraining cannot be established. Repeatedly inspected selections support regression testing; they cannot become unseen tests through rerunning.

The upstream dataset is licensed under Creative Commons Attribution-ShareAlike 4.0. Dataset-derived excerpts and retained outputs require the applicable attribution and licensing. The project keeps the corpus revision and licence with its evaluation materials.

6 Evaluation design

Whole-system comparison

The evaluation uses the released extraction code and two previously inspected samples: 20 development dialogues for all three methods and 20 test dialogues for FormBridge regression testing. Dialogue IDs were saved before the calls. Appendices A and B identify the source revision, configuration and selections.

All methods see identical ordered conversation prefixes and finish at the same customer turn. Direct extraction and FormBridge use GPT-6 Luna as the primary model. FormBridge also permits a Sol repair or recovery and requests quotations. This compares complete systems, with different prompts, output allowances and call counts.

Controlled component and feature checks

A second comparison uses one primary-model proposal for the final visible prefix of each development dialogue. Each proposal is scored before and after deterministic validation. Both conditions therefore share the exact input, prompt, model, output allowance and response. Repair and earlier predicted state are excluded. This isolates postprocessing of these proposals, rather than the complete incremental application. The one timed-out request counts as a failure in both conditions.

Twenty constructed conversations separately test corrections, customer acceptance, cancellation, dates, guest counts and uncertainty. Expected values, statuses, supporting-message combinations and clarification targets were written before calls and kept out of model inputs. Each complete transcript is analysed from an empty brief. The checks measure source coverage, preservation of constraints, question targets and approval decisions; they do not rate conversational helpfulness.

Metrics

Field accuracy is the number of matching final values divided by five times the number of attempted dialogues. For 20 dialogues the denominator is 100. **Complete-brief accuracy** is the number of dialogues with all five matching values divided by the number attempted. A failed dialogue remains in both denominators; staff edits are excluded.

The strict matcher applies basic text normalisation. The canonical matcher also accepts equivalent date and time forms, without supplying missing years or treating a range as an exact time. Both are reported for the whole-system comparison. They measure five-field matching, not full SGD joint goal accuracy or readiness to approve.

Resource accounting includes OpenAI requests, observed tokens and whole-dialogue replay time. Replay includes multiple prefixes, so it is not one-message response time. Failed requests may omit usage; token totals are not complete billing records.

Reproduction and interpretation

The scripts call application functions with real provider requests; separate browser checks exercise the deployed interface. Saved outputs retain predictions, failures, settings and source checksums. The main runner rechecks the extraction checksum before saving. The component and feature runner rechecks its combined source checksum. Hosted-model variability prevents identical future outputs even with unchanged source. Appendix E documents the supplementary checks.

7 Results and resource use

Development comparison

Table 3 reports the 20 development dialogues evaluated on 13 October 2026. Each method completed all 20; the field denominator is 100. Complete-brief scores require all five values to match within the same dialogue.

Table 3. Development accuracy on 20 dialogues.

Method	Strict fields	Canonical fields	Strict complete	Canonical complete
Rules	35/100	35/100	0/20	0/20
Direct Luna	83/100	89/100	6/20	11/20
FormBridge	97/100	97/100	17/20	17/20

FormBridge exceeded direct extraction by eight canonical field matches and six complete briefs on this selection. This is a difference between the configured systems, including prompts, output representation, reconciliation and additional repair calls. It cannot be attributed to validation alone. Both models found every restaurant and city; dates and guest counts account for most remaining differences.

Effect of validation on identical proposals

Table 4. Paired comparison using one proposal per final dialogue prefix.

Condition	Canonical fields	Complete briefs
Original model proposal	88/100	14/20
Same proposal after validation	85/100	12/20

The 20 attempts include one timeout, counted as five wrong fields in each condition. Validation gained one field match but lost four correct values. This result shows that the deterministic checks alone did not improve value accuracy on these proposals. Section 8 examines why; it is not a repeat of the full-system experiment.

Additional computation

In the whole-system comparison, each method processed 166 customer prefixes. Direct extraction made 166 OpenAI requests; FormBridge made 196. Observed input/output tokens were 93,947/21,817 and 381,706/61,309 respectively. Three FormBridge dialogues recovered from failed primary requests without returned usage, so their token totals are lower bounds.

Mean dialogue replay time was 21.87 seconds for direct extraction and 45.11 seconds for FormBridge. The latter obtained more matching values at approximately twice the replay time. Concurrent evaluation activity limits latency comparisons. Rules required no provider calls.

Inspected regression sample

On the inspected test selection, FormBridge matched 100/100 fields and 20/20 complete briefs under both matchers. It processed 137 prefixes with 149 provider attempts, 289,466 input and 50,732 output tokens. Mean replay time was 32.93 seconds, with no failed dialogues or incomplete usage records. This is regression evidence for the selected cases.

8 Error analysis and development findings

What the three remaining field mismatches mean

The saved development predictions were compared with their source conversations. Two failures concern how far a short acceptance should extend across earlier staff proposals. The third concerns the matcher rather than a different requested date.

Table 5. FormBridge development mismatches.

Dialogue / field	Observed output	Interpretation
1_00005 / date	Missing; label is “today”	Staff proposed today before venue corrections. The later accepted summary omitted the date; the earlier proposal was not retained.
1_00017 / guests	Missing; label is 2	Staff proposed two guests before date and venue changes. The accepted final summary omitted the count; it was not retained.
4_00069 / date	“the 5th of this month”; label is “5th of this month”	The date is retained as uncertain. The matcher does not remove the leading article from this relative phrase.

One mismatch is a text-equivalence issue. A broader relative-date normaliser should be developed separately, then applied to all methods alongside the existing metric.

The first two cases concern acceptance across intervening corrections. They count as errors against the dataset labels. Their full-system traces do not retain the original model proposals, so those runs alone cannot separate extraction from validation errors.

What the controlled comparison reveals

The paired run retains original proposals. In 4_00080, validation rejected San Francisco when the customer quotation said “San Fran”. In 4_00089, two correct values were rejected because their citations omitted earlier customer support. In 1_00005, the proposed date cited a staff summary without the required customer acceptance. These four losses show how strict matching and incomplete citations can discard correct values despite useful information elsewhere in the transcript.

Validation gained one match by retaining “Wednesday next week” as a value while keeping its uncertainty. That is a representation change, not newly recovered information. The whole-system result also partly reflects representation: six direct-model date mismatches retained the relative phrase only as a constraint. The constructed-case checks therefore assess uncertainty and approval separately from values.

Development lessons

The rule baseline matched no dates because it recognises full calendar dates while this selection contains relative or yearless forms. A broader date parser and trained slot tagger would make stronger baselines. For FormBridge, the next improvement is to recover missing supporting messages and recognise valid aliases without accepting unsupported values.

Earlier tests exposed negated cancellations, stale approvals after invalid corrections, disappearing retry controls and late replies appearing in another case. Cancellation checks, approval invalidation, persistent recovery controls and request scoping addressed those failures. Targeted tests and deployed retests support these fixes, without establishing a zero-error rate.

9 Interaction design

The main interaction is a conversation beside a booking card. Staff can inspect a field's source, revise a message or speaker, compare a changed value and ask for the unresolved detail. Status text accompanies colour. Approval is a separate action and is removed when the underlying request changes. This approach follows human–AI interaction guidance on explaining behaviour and supporting correction (Amershi et al., 2019).

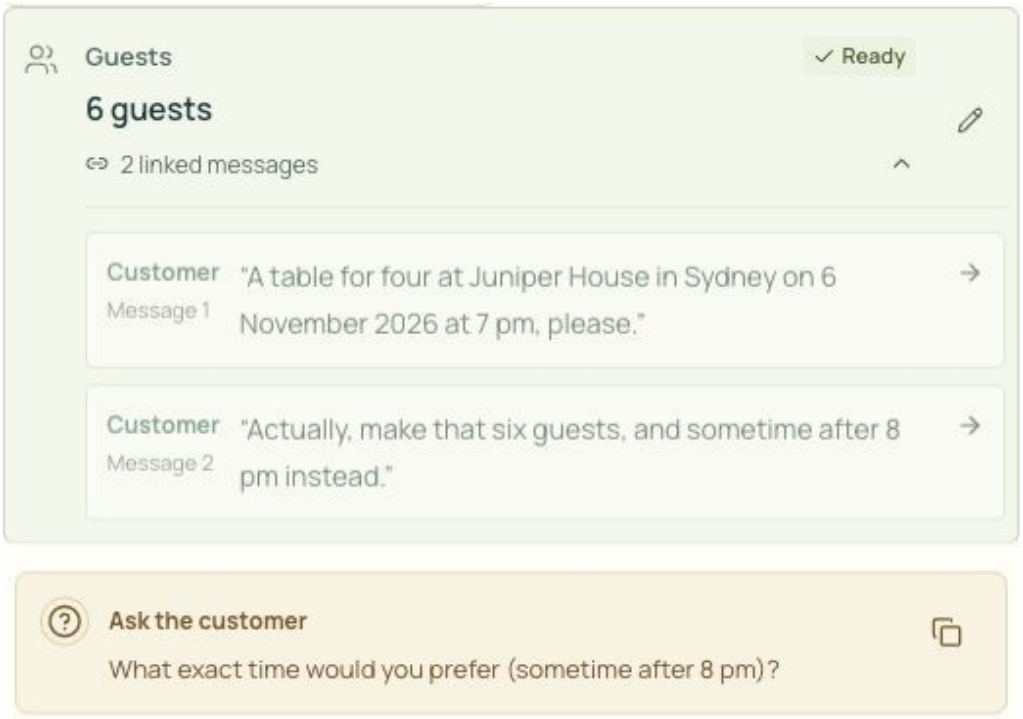


Figure 3. Enlarged source panel from Figure 1. Staff can read the original request and correction beside the six-guest value. The separate time question preserves the customer's “after 8 pm” preference.

Eight tasks completed on the live application

One structured walkthrough completed eight tasks: open a blank desk, add a complete enquiry, correct guests and time, inspect sources and history, propose a time, accept it, approve the brief, and edit an earlier message. All eight produced the expected visible state. History showed four guests becoming six and 7 pm becoming unresolved. A staff offer of 8:30 pm left approval blocked until the customer accepted. Editing the approved request then removed approval and rebuilt the affected count. Appendix F records the steps and history view.

Source inspection, visible changes and reversible editing make errors reviewable. Cream surfaces, dark green navigation and serif headings give the restaurant workspace a consistent identity; status text accompanies colour. The walkthrough establishes that these controls and transitions worked. It does not measure a new user's learning time, discoverability, satisfaction or restaurant productivity. No recruited participants were involved.

10 Functional verification

The released application passed 230 automated tests across 27 files, TypeScript checking and the production build. Separate production and browser records cover shared access, concurrency and complete workflows. Their original revisions are listed in Appendix C.

Table 6. Verification scope and results.

Evidence	Result and scope
Automated tests	230 passed: state, evidence, dates, input limits, request scoping and email-preview transitions
Production API/database	31/31 checks passed with real Supabase identities, permissions, shared records and routes
Restaurant browser workflows	22 recorded workflows: intake, consent, care, review, team work, widget and simulated handoff
Email preview	Local and deployed account flows completed; no real mail or production account created

The later email-preview release did not alter the previously checked restaurant extraction or backend paths. Counts are reported separately because the checks overlap.

Source, uncertainty and clarification checks

All 20 constructed conversations completed with the expected field values and statuses: 100/100 checks for each. There were 20 OpenAI requests and no final failures. The feature-specific results were:

Table 7. Feature checks on constructed conversations.

Check	Matched	Meaning
Required source messages	93/93 populated fields	Correct content included its prelabelled supporting-message combination
Uncertainty retained	7/7 uncertain fields	Expected status, value and constraint were preserved
Clarification target	9/9 required questions	Each question addressed the expected unresolved detail
Ask or remain silent	20/20 cases	Missing details triggered questions; complete or cancelled requests did not
Approval eligibility	20/20 cases	Approval matched the expected complete, incomplete or cancelled state

Source coverage does not score every extra quotation or independently judge wording. Clarification checks identify the requested field, not how helpful a person finds the question. These are constructed functional tests, separate from SGD accuracy and participant research.

Production checks also rejected cross-team access and stale edits, protected private review data and preserved enquiries after simulated provider failures. Selected desktop and mobile layouts showed no horizontal overflow. Keyboard checks covered dialog closing and focus return. These observations do not certify every browser or assistive technology.

11 Ethics and limitations

Guest information and access

Conversations can contain names, contact details and dietary requirements. Public intake asks the guest to consent before sharing contact details with the restaurant. Team membership restricts access to private records. PostgreSQL row-level security protects reads, and authorised mutations check the caller and record version (Supabase, n.d.). Private review links expire after 72 hours and expose only the information needed for guest review; anyone possessing an active link can use it.

OpenAI processes the submitted text or audio needed for a request. Responses API calls use `store: false`, which does not by itself guarantee zero provider retention; applicable abuse-monitoring and account controls still matter (OpenAI, n.d.-a). FormBridge does not retain raw voice recordings. JSON exports contain transcript data and should be handled as guest records.

The current product has no automated retention schedule for all shared enquiries or a complete data-subject request workflow. A restaurant deployment would need an agreed retention policy, deletion procedures and an explanation of external processing. Free-plan service limits and dependency availability also constrain operation.

Human decisions and consequences

An extracted allergy or accessibility request can be incomplete. Staff should confirm practical arrangements with the guest and restaurant; extraction does not establish medical or operational suitability. Returning-guest records are linked deliberately to avoid combining different people with the same name. Staff can reject suggestions, correct fields and inspect their sources.

Prompts and validation reduce the effect of instructions embedded in a conversation, but do not prove immunity to prompt injection. Rate limits and reserved usage reduce uncontrolled model spending, while failed requests preserve the last valid brief. These controls have bounded test evidence rather than an assurance against every malicious input or outage.

Evaluation and deployment limits

The evaluated domain is English restaurant enquiries. Twenty inspected dialogues per selection, one current run and hosted model aliases do not support broad accuracy or statistical superiority claims. Constructed cases measure supporting-message coverage, uncertainty and approval decisions, but provide no independent rating of every quotation or of clarification usefulness. The paired experiment exposes validation losses on realistic dataset conversations. Public-data pretraining contamination is unknown.

The guest endpoint currently uses Australia/Sydney as its reference-date timezone; reminder times follow the staff device. Restaurant-specific timezone configuration is needed before wider geographic deployment. Reminders update while the application is open; background notification delivery is absent. Device identities are tied to their browser and do not offer email recovery.

Messaging and reservation connectors are explicit simulations. Email registration and recovery have a separate simulation, while real delivery awaits a configured sender. There is no live reservation-provider integration, restaurant availability verification or payment processing. The application is a tested enquiry workflow with these stated operating boundaries.

12 Project delivery and next steps

Project phases and contribution

The project progressed through five phases, extending the initial booking brief into a shared restaurant enquiry workflow.

Table 8. Project phases and outputs.

Phase	Main output
Problem definition	Restaurant enquiry scope, current-state fields and a review-first workflow
NLP implementation	Rules, direct extraction, evidence validation, state reconciliation and repair limits
Evaluation and correction	SGD mapping, repeatable selections, strict and canonical scoring, error investigation and regression tests
Product development	Restaurant-styled interface, voice input, saved cases, team access, guest intake, care and review links
Release and communication	Deployed checks, reproducibility instructions, product films and technical findings

Karthik Ramesh (26043455) is responsible for technical implementation. This includes the application, NLP integration, evaluation tooling, deployment and technical verification.

Recommendations in priority order

1. **Test with restaurant staff.** Use paired tasks with and without FormBridge, vary task order and measure errors, completion time and reasons for corrections. Obtain consent and use fictional guest data.
2. **Improve evidence handling and evaluation.** Address the alias and missing-citation failures from the paired test. Reserve independent data before tuning, use a second annotator for source and clarification judgements, and strengthen the rule baseline. Isolate the contribution of repair in a further controlled comparison.
3. **Complete operational setup.** Configure and verify real account email delivery, recoverable staff access, restaurant timezones, retention and background reminders. Replace mock connectors only after provider authorisation and sandbox testing.
4. **Reduce waiting and cost.** Investigate when repair is needed and whether shorter evidence outputs preserve accuracy. Test both quality and latency before release.

Conclusion

FormBridge demonstrates an end-to-end NLP application for a defined user group. It connects contextual extraction to inspectable evidence, revision-aware approval and a shared restaurant enquiry process. On the current development selection it matched 97 of 100 fields and 17 of 20 complete briefs, exceeding the configured direct baseline while using more computation. The inspected regression selection matched all 100 fields.

The paired comparison shows that stricter validation can also discard correct values. Constructed cases and the live walkthrough demonstrate selected review behaviours; independent data and restaurant user studies should now test whether those behaviours improve enquiry handling.

References

Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S., Bennett, P. N., Inkpen, K., Teevan, J., Kikin-Gil, R., & Horvitz, E. (2019). Guidelines for human–AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (pp. 1–13). Association for Computing Machinery. https://doi.org/10.1145/3290605.3300233

Google Research. (n.d.). *Schema-Guided Dialogue dataset* [Data set; revision e852981ae34990f435897962585425930 2feaa78]. GitHub. https://github.com/google-research-datasets/dstc8-schema-guided-dialogue/tree/e852981ae34990f435897962585425930 2feaa78

OpenAI. (n.d.-a). *Data controls in the OpenAI platform*. Retrieved October 13, 2026, from https://developers.openai.com/api/docs/guides/your-data

OpenAI. (n.d.-b). *Structured model outputs*. Retrieved October 13, 2026, from https://developers.openai.com/api/docs/guides/structured-outputs

Rastogi, A., Zang, X., Sunkara, S., Gupta, R., & Khaitan, P. (2020). Towards scalable multi-domain conversational agents: The Schema-Guided Dialogue dataset. *Proceedings of the AAAI Conference on Artificial Intelligence, 34*(5), 8689–8696. https://doi.org/10.1609/aaai.v34i05.6394

Supabase. (n.d.). *Row level security*. Retrieved October 13, 2026, from https://supabase.com/docs/guides/database/postgres/row-level-security

Wu, C.-S., Madotto, A., Hosseini-Asl, E., Xiong, C., Socher, R., & Fung, P. (2019). Transferable multi-domain state generator for task-oriented dialogue systems. In A. Korhonen, D. Traum, & L. Màrquez (Eds.), *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (pp. 808–819). Association for Computational Linguistics. https://doi.org/10.18653/v1/P19-1078

Appendix A · Run the application

Source and environment

Source: https://github.com/KarthikRamesh9149/NLP-AT2

Live application: https://formbridge-nlp-at2.vercel.app/

Use Node.js 24, pnpm 11.24.0 and Git. The commands below select the exact application revision evaluated in this report. The public repository's later documentation commits contain this report and its saved results.

```
git clone https://github.com/KarthikRamesh9149/NLP-AT2.git
cd NLP-AT2
git checkout b0d389e410af735ba622b41a937adc351162058b
pnpm install --frozen-lockfile
cp .env.example .env.local
```

Edit `.env.local` in a text editor. Set `OPENAI_API_KEY` to a key from your own API project, with access to the configured models. Keep the key out of source control. Set `OPENAI_MODEL=gpt-6-luna` for the report's direct baseline; the template's older direct-model default is not the reported configuration. Keep `FORMBRIDGE_MODEL=gpt-6-luna` and `FORMBRIDGE_REPAIR_MODEL=gpt-6.1-sol`.

Shared features require a dedicated Supabase project. Apply every file in `supabase/migrations/` in filename order. Set the public project URL and publishable key, plus the server-only `SUPABASE_SECRET_KEY`. Enable anonymous sign-in in Supabase before setting `NEXT_PUBLIC_DEVICE_SIGN_IN_ENABLED=true`. Rebuild after changing public environment variables. Keep real email delivery disabled until a sender and the `/auth` redirect have been verified.

```
pnpm test
pnpm typecheck
pnpm build
pnpm start
```

Open `http://localhost:3000`. Keep this terminal running and use a second terminal, in the same directory, for route checks or dataset commands. Stop and restart the server after rebuilding. `pnpm dev` supports development without shared storage; production AI routes deliberately reject requests when the shared allowance database is absent.

Verification and provider boundaries

The repository's `docs/OPERATIONS.md` describes workspace setup, permissions, usage allowances and connector simulations. `scripts/check-restaurant-workspace.ts` creates synthetic accounts and cases against a configured test project. It needs the Supabase variables; use a dedicated project and retain its output to identify test records. Real API runs consume provider usage.

The `/auth/preview` route requires no mail sender. It accepts sample `.test` addresses, keeps its account and inbox in memory and clears them on refresh. Its verification and recovery links expire after 15 minutes and can be used once. This simulation has no access to real team records.

Appendix B · Reproduce the evaluation

Dataset and configuration

Run these commands from the application directory in a second terminal. Dataset acquisition is separate from installing the app. Preserve the upstream Creative Commons Attribution-ShareAlike 4.0 licence and attribution with dataset-derived material.

```
git clone \
  https://github.com/google-research-datasets/dstc8-schema-guided-dialogue.git \
  ../formbridge-sgd
git -C ../formbridge-sgd checkout e852981ae34990f4358979625854259302feaa78
pnpm data:audit ../formbridge-sgd

export OPENAI_MODEL=gpt-6-luna
export FORMBRIDGE_MODEL=gpt-6-luna
export FORMBRIDGE_REPAIR_MODEL=gpt-6.1-sol
```

The two selection files below already exist at application revision `b0d389e` and contain exactly the IDs used by the refreshed runs. The runner copies them into new selection records before making provider calls. Gold values are only used for scoring.

```
EVAL_RESULTS=evaluation/results
pnpm exec tsx --env-file=.env.local scripts/evaluate.ts ../formbridge-sgd \
  --split=dev --sample=20 --transport=in-process \
  --replay=all-user-turns --modes=rules,model,formbridge \
  --selection="$EVAL_RESULTS/selection-product-v17-20261013-same-primary-development.json" \
  --tag=reproduce-report-development

pnpm exec tsx --env-file=.env.local scripts/evaluate.ts ../formbridge-sgd \
  --split=test --sample=20 --transport=in-process \
  --replay=all-user-turns --modes=formbridge \
  --selection="$EVAL_RESULTS/selection-product-v17-20261013-regression.json" \
  --tag=reproduce-report-regression
```

Saved outputs and scoring

The report uses `sgd-dev-20-report-v18-20261013-development.json` and `sgd-test-20-report-v18-20261013-regression.json`, under `evaluation/results/` on the report revision. Each includes selected IDs, model settings, per-field predictions, final states, per-turn usage and summary metrics. Corresponding `selection-report-v18-...` and `progress-report-v18-...` files preserve selection and progress.

Use a new tag for another run: the runner refuses to overwrite selections or final results. Keep all attempts, including provider failures and lower scores. A rerun of these samples remains development or regression evidence.

The extraction version is `grounded-state-v18`. Its source checksum is `b9c32b2629269567a2fba24edd706bd7fdc102df7210c6721693b28f2a515424`; the runner checksum is `1b89aeedce527eaec164b0530f98d3072709009c57def1162fadedd40a6920ef`. The runner records both but rechecks only the extraction checksum before saving results. Matching uses `lib/evaluation-match.ts`; prefix replay uses `lib/evaluation-replay.ts`. Unchanged files do not guarantee identical hosted-model responses.

Appendix C · Evidence and comparisons

Current evidence records

All paths below are relative to the public repository. Saved results identify their original source version and scope. Each record preserves the inputs, observations or instructions needed to audit the reported findings.

Table C1. Retained technical evidence.

Record	Scope
`evaluation/results/sgd-dev-20-report-v18-20261013-development.json`	Current three-method comparison: 20 development dialogues, 100 final field checks per method
`evaluation/results/sgd-test-20-report-v18-20261013-regression.json`	Current FormBridge regression: 20 inspected test dialogues
`evaluation/results/restaurant-workspace-production.json`	31 real production route and database checks at `0b25ba8`; provider actions simulated
`evaluation/results/restaurant-workspace-browser.md`	22 restaurant workflows and associated edge checks; final source `ddf8acd`
`evaluation/results/email-preview-verification.md`	Email simulation tests and deployed verification at `b0d389e`
`tests/`	Automated test inputs and assertions; 230 passing at `b0d389e`

Historical equal-model comparison

The earlier application at `32af459b6a1cd20ede9b08c2b73c20a0c385301e` compared Luna direct extraction with Luna primary and Luna repair in FormBridge. On 20 development dialogues, both reached 92/100 canonical fields; FormBridge matched 14/20 complete briefs. On a further 13 then-unselected eligible test dialogues, both contextual methods matched 62/65 fields and 12/13 complete briefs.

These results qualify the interpretation of the new comparison. The earlier system did not obtain a canonical field advantage under that equal-model configuration. The current application, prompts and repair model differ, so comparing the two releases cannot attribute improvement to a single component. Even equal-model runs may have different numbers of calls and output budgets.

The historical files and commands remain in `evaluation/REPRODUCIBILITY.md` with their source revision. They are not pooled with the current 20-dialogue results and are not described as current unseen performance.

Traceability of figures

Figures 1 and 3 are different crops of the same unaltered live capture, `docs/assessment/assets/FormBridge-core-workflow.jpg`. It shows the fictional enquiry used in the eight-task walkthrough. Figure 4 is the actual guest review capture retained in that asset directory. Figure 2 is drawn from the implemented request and storage paths.

Appendix D · Guest review workflow

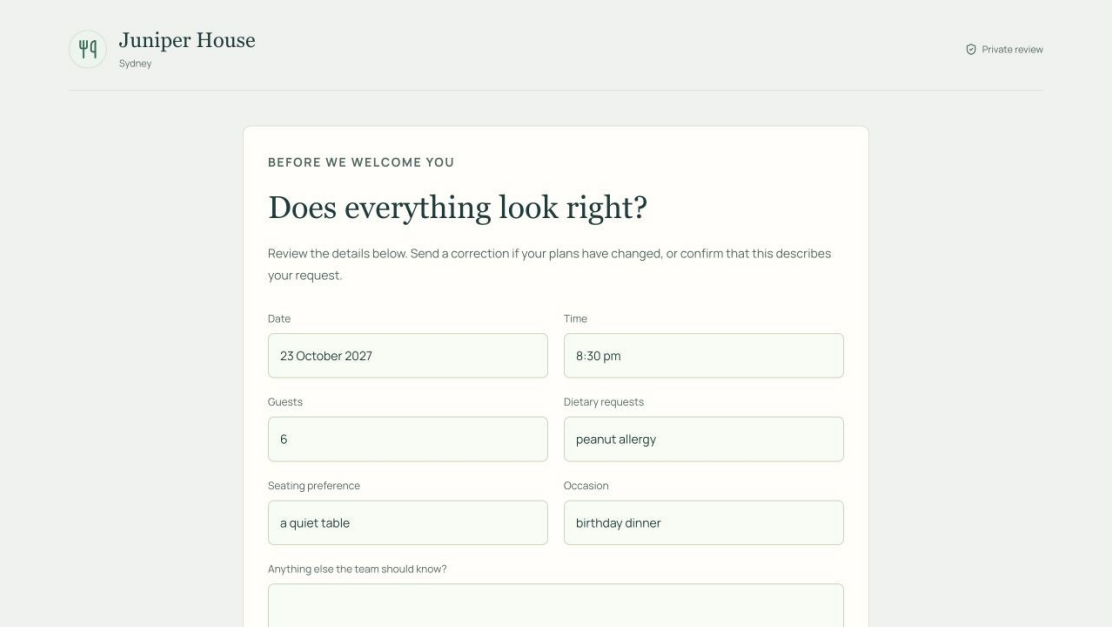


Figure 4. The deployed guest review page with a fictional enquiry. A private link presents the booking details for confirmation or correction; it does not expose the staff conversation or reserve a table.

State transitions checked in the product

Table D1. Guest review and handoff transitions.

Event	Result
Staff approve a current brief	Approval applies to that revision; a saved case can receive a private review link
Guest opens a valid link	Only the selected booking details and preferences are returned
Guest requests six places instead of five	A new customer message is queued; approval is removed and the case returns to Needs reply
Staff check the correction	The count updates while the retained date and time remain visible for review
Link is revoked or expired	The page is unavailable and reveals no saved snapshot
An unsubmitted link refers to an older case revision	The page is unavailable; staff must issue a current link
A submitted review is reopened	Its receipt remains accessible while the link and case remain valid
Simulated provider fails	The approved enquiry remains available for staff action

The review flow demonstrates how language input becomes an accountable change to a shared record. It also shows why a correct final value is only one part of the product: the change must reach the right enquiry, invalidate the right approval and remain understandable to staff and guest.

Appendix E · Supplementary experiments

Model-selection trials

Table E1. Historical trials on the same ten development dialogues.

Configuration	Strict fields	Complete	Failed	Mean replay
Luna primary, version 2	45/50	5/10	0/10	27.78 s
Sol primary, Astra repair, version 3	38/50	4/10	1/10	52.39 s

The second mean covers nine completed dialogues. Its failed request returned HTTP 503. The files are `sgd-dev-10-grounded-luna-pilot.json` and `sgd-dev-10-grounded-v3-development.json` in `evaluation/results/`. Prompt and validation changes prevent a causal comparison of models alone; these trials inform a practical configuration choice.

Reproduce the paired and feature checks

After the setup in Appendix A, select the revision containing the supplementary scripts. The application code is unchanged from the evaluated release. Keep the same model variables and dataset checkout. Run each command with a new tag to preserve earlier results.

```
git checkout 470c81f883065eb0a98dba7bfb7d9e05511043b5
pnpm install --frozen-lockfile
pnpm exec tsx --env-file=.env.local scripts/report-evidence.ts \
  component ../formbridge-sgd --tag=reproduce-component
pnpm exec tsx --env-file=.env.local scripts/report-evidence.ts \
  features --tag=reproduce-features
```

The paired test reuses the main development selection, makes one primary call for each final prefix and compares that proposal with its validated state. It excludes repair and previous predictions. Failed requests stay in the denominator. The feature test uses the 20 transcripts and expected states in `evaluation/report-feature-cases.ts`. Source coverage requires a correct populated field and one complete prelabelled combination of supporting messages. The question check matches the expected target; approval calls the application's approval function.

Outputs beginning `report-component-20261013-audit-followup` and `report-features-20261013-audit-followup` contain the saved protocols, progress and final predictions in `evaluation/results/`. Both runs checked the combined source checksum before and after execution. The script records the pre-commit revision; the command above selects the committed script and unchanged application sources.

The live walkthrough is recorded in `evaluation/results/report-interface-walkthrough-20261013.json`. It documents eight observable tasks in one fictional enquiry, with the screenshot used for Figures 1 and 3. It contains no participant ratings or time-saving estimates.

Appendix F · Booking-desk walkthrough

The deployed booking desk was exercised with a fictional enquiry for Juniper House in Sydney on 6 November 2026. Figure 5 shows the correction history from the same capture as Figures 1 and 3.



Figure 5. The expanded history keeps the previous values visible. A revised preference removes the exact time without erasing the other booking details.

Table F1. Observed tasks in one live enquiry.

Task	Observed result
1. Open a blank desk	No sample loaded; zero of five details ready and approval disabled
2. Add a complete request	Restaurant, city, date, 7 pm and four guests shown with sources
3. Change guests and time	Six guests retained; “after 8 pm” left unresolved; approval disabled
4. Inspect sources and history	Original and correction messages visible; superseded values shown
5. Propose 8:30 pm as staff	The offer alone did not resolve the time or enable approval
6. Accept 8:30 pm as customer	Five details ready; time resolved with acceptance evidence
7. Approve the brief	Approved state and download control shown; no reservation claimed
8. Edit the earlier guest count	Count rebuilt as five; other details retained and approval removed

All eight tasks reached the expected state. This is a functional walkthrough of one conversation, not eight participants or a usability study. It demonstrates the correction and review path; a participant study would need separate measures of discoverability, completion time, mistakes and feedback.